SQL 学习

写在前面：我的学习宣言

我终于下定决心，要系统地征服 SQL 了。过去，总觉得 SQL 就是 `SELECT * FROM table`，简单但又似乎深不可测。

这本日记，就是我这场学习之旅的忠实记录。它不仅是知识点的堆砌，更是**我（一个非科班出身、希望用数据武装自己的产品/项目人）**的思考、练习、踩坑与复盘的集合。我的目标很明确：用 12 周的时间，从一个只会 `SELECT` 的“小白”，成长为一个能独立完成复杂数据分析、理解数据库原理、并最好能进行性能调优的“准专家”。

这不仅仅是一份学习计划，这是我与数据深度对话的开始。旅途或许漫长，但每一步都会留下坚实的脚印！

我的 12 周 SQL 征服路线图

我为自己制定了一个为期 12 周的“爬山虎”计划，从最基础的墙角开始，一步步攀上 SQL 这座高墙的顶端。

周次	核心主题	我要达成的目标	关键里程碑
W1	**启程！SQL 与我的第一次亲密接触**	掌握基本 `SELECT` 查询，理解关系模型，能写简单的过滤、排序和聚合。	完成第一次独立的数据查询与分析任务。
W2	**深入查询的艺术——JOIN 与聚合**	精通各种 `JOIN`，掌握 `GROUP BY` 和子查询，能处理多表关联数据。	能独立完成跨 3 个以上核心业务表的复杂数据提取。
W3	**事务与并发，保证数据的小船不翻**	理解 ACID、隔离级别和 MVCC，知道什么是脏读、幻读，能写基本的事务代码。	完成一个模拟并发场景下数据问题的复现与修复实验。
W4	**索引与优化：给我的查询安上火箭**	明白索引为什么能加速，掌握 `EXPLAIN`，能分析并优化慢查询。	成功将一个耗时超过 10 秒的查询优化到 1 秒以内。
W5	**高级语法：窗口函数与 CTE**	熟练使用窗口函数进行排名、分析；掌握 CTE 进行	完成一个复杂的业务排名（如“各品类 TOP 5 商

		复杂逻辑拆解和递归查询。	品”）。
W6	**数据建模与项目实践（一）：电商分析**	从需求到 ER 图，完成一个小型电商数据模型的物理设计和脚本编写。	提交一份完整的电商销售分析报告（SQL 驱动）。
W7	**玩转半结构化与特殊数据**	学会用 SQL 处理 JSON、XML；了解 PostGIS 处理地理空间数据。	能从 JSON 日志中提取关键字段并进行聚合分析。
W8	**OLAP 与数仓的世界观**	理解 OLTP 与 OLAP 的区别，掌握星型/雪花模型，了解 SCD。	设计一个迷你的“用户行为分析”数据宽表。
W9	**云数仓与新一代引擎巡礼**	了解 Snowflake, BigQuery, ClickHouse, DuckDB 等的特点与适用场景。	在至少一个云数仓/新引擎上完成一次数据查询与体验。
W10	**工程与可靠性：从 SQL 到数据工程**	学习幂等、数据质量校验、流式 SQL (Flink SQL) 的基本概念。	编写一个幂等的 `UPSERT` 脚本。
W11	**项目实践（二）：日志告警系统**	设计并实现一个简单的日志数据分析与告警系统，练习时序聚合。	完成一个“连续 5 分钟错误率超 5%”的告警 SQL。
W12	**毕业总结：跨库差异与我的知识体系**	系统梳理各主流数据库的语法、功能差异与迁移要点，形成自己的知识图谱。	完成一篇“我的 SQL 学习之路”复盘，并创建个人 SQL 工具箱。

第一周：启程！SQL 与我的第一次亲密接触

学习目标

1. **建立认知**：理解数据库和 SQL 是什么，它们如何协同工作。
2. **核心语法**：掌握 `SELECT` 语句的核心组件：`FROM`，`WHERE`，`ORDER BY`，`LIMIT`。
3. **简单聚合**：学会使用 `COUNT`，`SUM`，`AVG`，`MAX`，`MIN` 对数据进行初步的统计。
4. **动手实践**：连接到一个真实的数据库，并写出我的第一条有意义的查询。

知识卡片

什么是数据库 (Database)?

可以把它想象成一个**结构化的数据仓库**。它不是一堆杂乱无章的文件，而是一个按照特定规则（比如关系模型）组织起来的、可以被高效存取和管理的电子数据集合。

- **我之前的误解**：以为 Excel 文件就是数据库。
- **现在的理解**：Excel 是数据，但管理大量、复杂、需要多人同时访问的数据时，就需要专业的数据库管理系统（DBMS）了。

SQL vs. 数据库 vs. DBMS

- **数据库 (DB)**：存放数据的仓库。
- **数据库管理系统 (DBMS)**：管理仓库的软件，比如 MySQL, PostgreSQL, Oracle 等。
- **SQL**：我和 DBMS 沟通的语言。我用 SQL 告诉 DBMS 我想要什么数据，DBMS 就会去仓库里帮我找出来。

这是一个“人 - 翻译器 - 图书管理员 - 图书馆”的模型。

我的第一个 `SELECT` 查询

这是 SQL 世界的 "Hello, World!"。它的结构就像说一句自然语言。

代码块

```
SELECT
    column1, -- 我想看的列
    column2
FROM
    table_name -- 从哪个表里看
WHERE
    condition; -- 在满足什么条件的情况下看

```

我的练习记录 1.1：查询用户表

假设我有一个 `users` 表，我想找到所有来自“北京”且注册时间在 2025 年之后的用户，并按注册时间倒序排列，只看前 10 个。

代码块

```
-- 我的第一条“正式”查询
SELECT
    user_id,
    user_name,
    email,
    registration_date
FROM
    users
```

```
WHERE
    city = '北京' AND registration_date >= '2025-01-01'
ORDER BY
    registration_date DESC -- DESC 表示降序，ASC 是升序
LIMIT 10; -- 只返回前 10 条记录

```

我的结果：（此处应为查询返回的表格结果，我在实际练习时会贴在这里）

我的复盘：

- `WHERE` 子句就像一个过滤器，可以组合多个条件，用 `AND` (与)，`OR` (或)。
- `ORDER BY` 是排序器，决定了结果的展现顺序。
- `LIMIT` 是截断器，对于快速预览数据非常有用。

我的第一次数据聚合

聚合函数是 SQL 的一大魅力，能快速地对海量数据进行总结。

函数	作用	我的理解
`COUNT(*)`	计算行数	“总共有多少条记录？”
`COUNT(column)`	计算某列非空值的数量	“有多少用户填写了邮箱？”
`SUM(column)`	求和	“所有订单的总金额是多少？”
`AVG(column)`	求平均值	“用户的平均年龄是多少？”
`MAX(column)`	求最大值	“最贵的商品卖多少钱？”
`MIN(column)`	求最小值	“最早的注册用户是哪天来的？”

我的练习记录 1.2：统计用户数据

我想知道 `users` 表里总共有多少用户，以及北京用户的平均年龄。

代码块

```
-- 聚合查询初体验
SELECT
    COUNT(*) AS total_users, -- 使用 AS 给结果列起一个别名，更易读
    AVG(age) AS avg_age_beijing
FROM
    users
WHERE
    city = '北京';
```

```
9
```

我的结果：

total_users	avg_age_beijing
15234	28.5

我的复盘：

- 聚合函数会把多行数据压缩成一行结果。
- `AS` 是个好习惯，能让结果的可读性大大提升。

第一周复盘与常见坑

- **易错点**：`WHERE` 子句中，字符串值需要用**单引号** `' '` 括起来，这和很多编程语言用双引号不同。
- **常见坑**：`NULL` 是个特殊的值，它不等于任何东西，甚至不等于它自己。所以 `WHERE column = NULL` 是错的，必须用 `WHERE column IS NULL` 或 `WHERE column IS NOT NULL`。
- **我的思考**：SQL 的逻辑顺序和书写顺序不一样。引擎实际执行时，大致是 `FROM` -> `WHERE` -> `SELECT` -> `ORDER BY` -> `LIMIT`。理解这个内部顺序，对以后理解复杂查询和优化很有帮助。

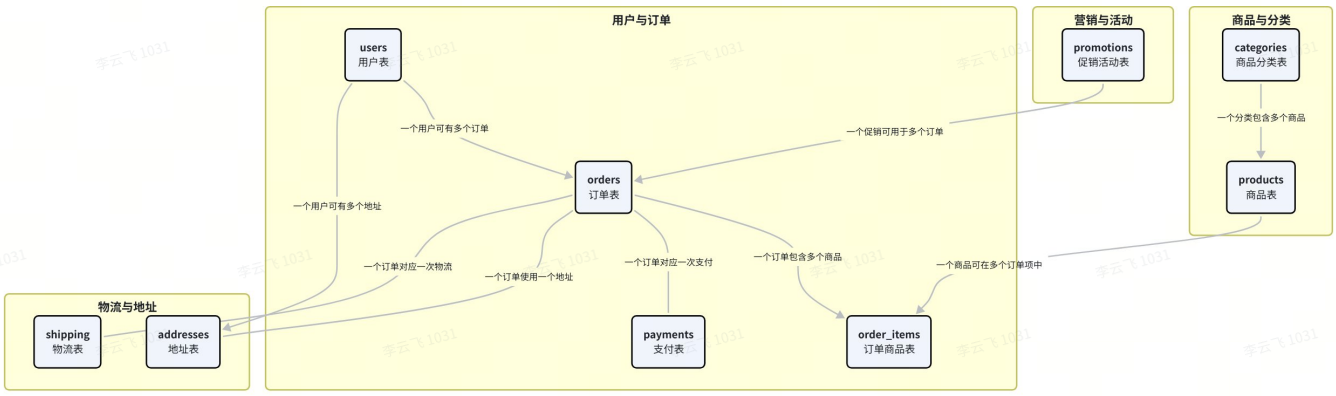
第二周：深入查询的艺术——JOIN 与聚合

学习目标

1. **精通 `JOIN`**：彻底搞懂 `INNER JOIN`，`LEFT JOIN`，`RIGHT JOIN`，`FULL OUTER JOIN` 的区别和用法。
2. **分组聚合 `GROUP BY`**：学会对数据进行分组，并对每个组进行聚合计算。
3. **分组过滤 `HAVING`**：掌握 `HAVING` 和 `WHERE` 的区别。
4. **子查询**：学习在查询中嵌套其他查询，处理更复杂的逻辑。

知识卡片：`JOIN` 的奥义

如果说上周的 `SELECT` 是在单张表里“寻宝”，那 `JOIN` 就是在多张藏宝图之间“按图索骥”。



（我的思考：这张 ER 图清晰地展示了 `orders` 表如何通过 `user_id` 与 `users` 表关联，通过 `product_id` 与 `products` 表关联（通过中间表 `order_items`），这是理解 `JOIN` 的基础。）

JOIN 类型	作用	我的比喻
`INNER JOIN`	只返回两个表中能**完全匹配**上的行。	**取交集**。像相亲，男女双方都得到场才算一次成功的会面。
`LEFT JOIN`	返回左表的所有行，以及右表中能匹配上的行。右表匹配不上的部分用 `NULL` 填充。	**左表优先**。以左表（`FROM` 后面的第一个表）为准，不管右边有没有人来，左边的人员名单必须全在。
`RIGHT JOIN`	和 `LEFT JOIN` 相反，返回右表的所有行。	**右表优先**。
`FULL OUTER JOIN`	返回左表和右表的所有行，互相匹配不上的部分都用 `NULL` 填充。	**取并集**。把两边的所有人都请来，能配对的坐一起，不能的也各自单坐。

我的练习记录 2.1：查询用户的订单

我想查询 `users` 表中名为“李云飞”的用户的**所有订单**信息。

代码块

```
-- 使用 INNER JOIN 查询特定用户的订单
SELECT
    u.user_name,
    o.order_id,
    o.order_date,
    o.total_amount
FROM
    users u -- 给表起别名，让查询更简洁
INNER JOIN
```

```
    orders o ON u.user_id = o.user_id -- ON 子句是连接的“桥梁”
WHERE
    u.user_name = '李云飞';

```

我的复盘：

- `ON` 子句是 `JOIN` 的灵魂，它定义了两个表如何关联起来。
- 给表起别名（如 `users u`）是最佳实践，能极大简化复杂查询的编写。
- `LEFT JOIN` 在什么场景下有用？比如，我想查询**所有**用户的订单情况，**即使某个用户没有下过单**。这时，如果用 `INNER JOIN`，没有订单的用户就不会出现在结果里。

代码块

```
-- 使用 LEFT JOIN 查询所有用户及其订单（即使没有订单）
SELECT
    u.user_name,
    o.order_id,
    o.order_date
FROM
    users u
LEFT JOIN
    orders o ON u.user_id = o.user_id;

```

对于没有下过单的用户，`order_id` 和 `order_date` 列就会显示为 `NULL`。

知识卡片：`GROUP BY` 与 `HAVING`

`GROUP BY` 是聚合查询的进化版。它能把数据按某个（或某几个）列的相同值“打包成组”，然后对每个组分别执行聚合函数。

`WHERE` vs `HAVING`

子句	作用阶段	作用对象	我的理解
`WHERE`	**分组前**过滤	原始的、未聚合的行	**入场券检查**：不符合条件的行，连参与分组的资格都没有。
`HAVING`	**分组后**过滤	`GROUP BY` 之后形成的“组”	**分组后筛选**：对已经打包好的小组进行二次筛选。

我的练习记录 2.2：统计每位用户的订单总额

我想计算每个用户的订单总数和总消费金额，并且只看那些消费总额超过 1000 元的用户。

代码块

```sql
SELECT
    u.user_name,
    COUNT(o.order_id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM
    users u
JOIN
    orders o ON u.user_id = o.user_id
GROUP BY
    u.user_name -- 按用户名进行分组
HAVING
    SUM(o.total_amount) > 1000 -- 对分组后的结果（每个用户的总消费）进行过滤
ORDER BY
    total_spent DESC;

```

我的复盘：

- `GROUP BY` 的威力在于“分而治之”，是统计分析的利器。
- `HAVING` 必须和 `GROUP BY` 一起使用（逻辑上），它操作的对象是聚合函数的结果。
- `SELECT` 子句中出现的非聚合函数列，都必须出现在 `GROUP BY` 子句中。这是个严格的规则，初学者很容易犯错。

第二周复盘与常见坑

- **`JOIN` 的性能陷阱**：在没有索引的列上进行 `JOIN`，或者 `JOIN` 的条件非常复杂，都可能导致查询性能急剧下降。这是第四周要深入研究的重点。
- **`COUNT(*)` vs `COUNT(column)`**：`COUNT(*)` 会统计所有行，包括 `NULL`。而 `COUNT(column)` 只统计该列值不为 `NULL` 的行。在 `LEFT JOIN` 的场景下，这个区别尤为重要。
- **我的思考**：SQL 的声明式特性在这里体现得淋漓尽致。我只需要告诉数据库“我要按什么分组，统计什么，过滤什么”，而不需要关心它内部具体是如何实现分组和聚合的（比如用哈希表还是排序）。但未来要优化，就必须了解这些内部机制。

第三周：事务与并发，保证数据的小船不翻

学习目标

1. **理解 ACID**：将 ACID 四个黄金准则刻在脑子里。

2. **并发异常**：能清晰地解释脏读、不可重复读、幻读这三种并发“鬼故事”。
3. **隔离级别**：掌握四种隔离级别，并了解主流数据库（特别是 MySQL 和 PostgreSQL）的默认级别和差异。
4. **MVCC 初探**：初步理解多版本并发控制（MVCC）是如何实现“读写不冲突”的。
5. **动手实验**：尝试用两个终端模拟并发，复现一个并发问题。

知识卡片：ACID，事务的基石

ACID 是什么？ 它是数据库事务必须遵守的四个特性，是数据可靠性的保证。

- **A (Atomicity) 原子性：要么全做，要么全不做。**转账操作，扣款和存款必须是一个整体，不能只发生一半。
- **C (Consistency) 一致性：数据总是对的。**事务前后，数据都必须满足所有预设的规则（如账户余额不能为负）。C 是目标，A、I、D 是手段。
- **I (Isolation) 隔离性：事务之间互不干扰。**我在查账时，不应该看到别人正在转账的中间状态。
- **D (Durability) 持久性：提交了就不会丢。**一旦事务成功，就算数据库断电重启，修改也得在。

并发异常：数据库里的“灵异事件”

想象一下，多个人同时操作一个共享文档，如果没有版本控制，会发生什么？数据库也是一样。

异常现象	描述	我的比喻
脏读 (Dirty Read)	读到了别人**还没提交**的数据。	我看到了朋友草稿箱里一篇还没写完、可能随时删除的文章。
不可重复读 (Non-Repeatable Read)	同一事务内，两次读**同一行**数据，结果不一样。	我刚看了股价是 10 元，刷新一下，变成了 11 元。数据**变了**。
幻读 (Phantom Read)	同一事务内，两次**范围查询**，第二次多出了一些“幻影”行。	我刚统计了部门有 10 个员工，再一统计，变成了 11 个。数据**多了**。

我的注意点：不可重复读 vs 幻读

- **不可重复读**是“行”的内容变了（`UPDATE` / `DELETE`）。
- **幻读**是“行”的数量多了（`INSERT`）。在 MySQL 的 InnoDB 引擎中，默认的“可重复读”级别已经通过 **Next-Key Locking** 技术很大程度上避免了幻读，这一点比标准定义更强。

隔离级别：在性能与一致性之间走钢丝

为了解决并发异常，SQL 标准定义了四种隔离级别，像四个档位，越高越安全，但可能也越慢。

隔离级别	脏读	不可重复读	幻读	主流数据库默认
读未提交 (Read Uncommitted)	可能	可能	可能	几乎不用
读已提交 (Read Committed)	❌	可能	可能	**PostgreSQL, Oracle, SQL Server**
可重复读 (Repeatable Read)	❌	❌	可能	**MySQL (InnoDB)**
可串行化 (Serializable)	❌	❌	❌	性能要求不高、一致性要求极高的场景

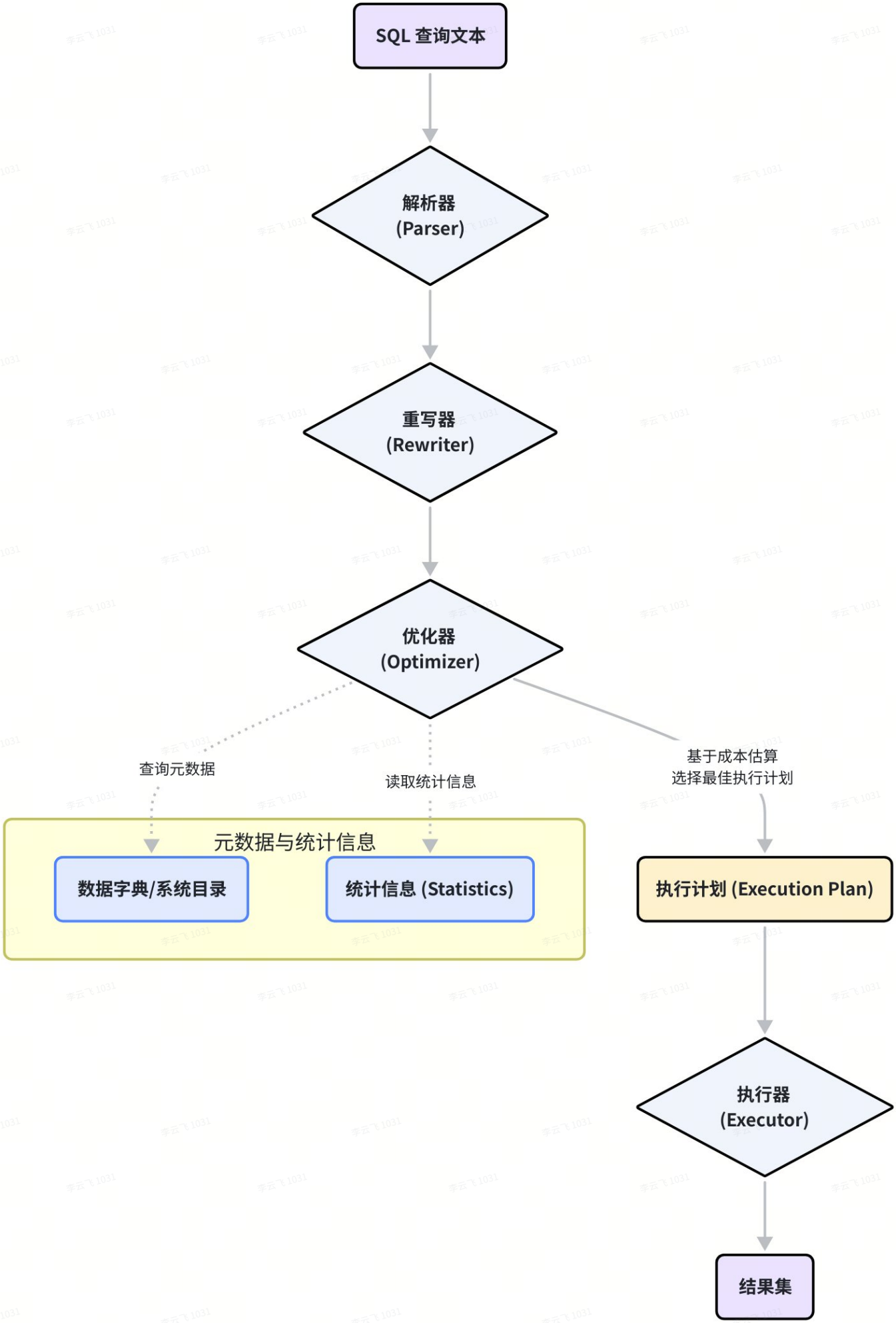
我的思考：为什么 MySQL 的默认级别比其他几个要高？这可能反映了其设计哲学更倾向于为开发者提供开箱即用的强一致性保证，避免了在“读已提交”级别下需要应用层代码来处理不可重复读的麻烦。

新特性摘要：PostgreSQL 17 与隔离级别我查阅了资料，虽然 PostgreSQL 17 在性能、JSON 处理等方面有巨大提升，但在核心的事务隔离级别定义上，仍然遵循 SQL 标准，默认级别依然是“读已提交”。[17] 这说明隔离级别的稳定性是数据库设计的核心，不会轻易变动。

MVCC：让“读”和“写”和平共处的智慧

MVCC（多版本并发控制）是现代数据库实现高效并发的秘密武器。

- **核心思想：以空间换时间**。不直接覆盖旧数据，而是为每一次修改创建一份**新版本**的数据。这样，不同的事务就可以根据自己的启动时间，看到不同版本的数据快照。
- **带来的好处：读写不加锁！** 读操作去读它该读的旧版本，写操作去创建新版本，两者互不干扰。这极大地提升了数据库的并发处理能力。


SQL 查询文本
解析器
(Parser)
重写器
(Rewriter)
优化器
(Optimizer)
查询元数据
读取统计信息
基于成本估算
选择最佳执行计划
元数据与统计信息
数据字典/系统目录
统计信息 (Statistics)
执行计划 (Execution Plan)
执行器
(Executor)
结果集

（复盘：这张图虽然是查询管线，但可以类比事务的执行。每个事务都像一个独立的查询请求，数据库内部的优化器和执行器会通过 MVCC 等机制，确保它们能高效、正确地并发执行，而不会互相“撞车”。）

我的练习记录 3.1：模拟“不可重复读”

我在 PostgreSQL（默认“读已提交”）下进行这个实验。

准备工作：创建一个简单的账户表。

代码块

```
CREATE TABLE accounts (
    id INT PRIMARY KEY,
    user_name VARCHAR(50),
    balance DECIMAL(10, 2)
);
INSERT INTO accounts VALUES (1, '李云飞', 1000.00);

```

实验步骤：

时间	终端 A (我的查询事务)	终端 B (模拟他人修改)	现象
1	`BEGIN;`		开启我的事务
2	`SELECT balance FROM accounts WHERE id = 1;` 结果: `1000.00`		第一次查询余额
3		`BEGIN;`	对方开启事务
4		`UPDATE accounts SET balance = balance - 200 WHERE id = 1;`	对方扣款 200
5		`COMMIT;`	**对方提交事务**
6	`SELECT balance FROM accounts WHERE id = 1;` 结果: `800.00`		**不可重复读发生！** 在我的同一个事务里，两次查询结果不同。
7	`COMMIT;`		结束我的事务

我的复盘：

- 这个实验清晰地展示了在“读已提交”级别下，一个事务可以看到另一个已提交事务的修改。
- 如果想避免，我可以将我的事务级别设置为“可重复读”：`BEGIN; SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`。这样，在我的事务结束前，无论别人如何提交修改，我看到的都会是事务开始时的那个“快照”，即余额一直是 `1000.00`。

第三周复盘与常见坑

- **事务忘记提交/回滚**：一个开启了但未结束的事务，会一直持有它占用的资源（比如锁），这可能导致其他事务长时间等待，甚至引发数据库连接池耗尽。**我的 Checklist**：每个 `BEGIN` 都必须对应一个 `COMMIT` 或 `ROLLBACK`。
- **大事务 (Long Transaction)**：在事务中包含用户交互、网络请求等耗时操作，是绝对的禁忌！这会让事务持有锁的时间变得极长，极大地影响系统并发性能。
- **我的思考**：事务不仅是数据库的概念，更是应用设计的一部分。何时开始事务、事务的边界在哪里、如何处理事务失败的重试，都需要在业务代码层面进行精心的设计。这周的学习让我意识到，数据库知识和后端开发密不可分。

第四周：索引与优化：给我的查询安上火箭

学习目标

1. **理解索引原理**：彻底明白为什么索引能让查询变快，理解 B-Tree 索引的结构和工作方式。
2. **掌握 `EXPLAIN`**：学会阅读 `EXPLAIN` 和 `EXPLAIN ANALYZE` 的输出，把它当作诊断慢查询的“听诊器”。
3. **索引类型大观**：了解不同类型的索引（B-Tree, Hash, GiST, GIN, BRIN）及其适用场景。
4. **索引失效的坑**：识别并避开那些会导致索引失效的常见写法。
5. **动手优化**：找到一个慢查询，并通过添加合适的索引或改写 SQL 来显著提升其性能。

知识卡片：`EXPLAIN`，我的SQL性能诊断书

`EXPLAIN` 命令是揭开数据库查询优化器“黑盒”的钥匙。它不会真正执行查询，而是告诉我数据库**计划**如何执行这个查询。

`EXPLAIN` 关键输出列	含义	我需要关注什么?
`type`	访问类型/连接类型	**性能好坏的关键指标！** 至少要达到 `range`，最好是 `ref` 或 `const`。`ALL`（全表扫描）是灾难。

`possible_keys`	可能用到的索引	优化器认为有哪些索引是备选。
`key`	**实际**用到的索引	优化器最终选择了哪个索引。如果没有，就是 `NULL` 。
`rows`	预估扫描的行数	这个数字越小越好。
`Extra`	额外信息	非常重要！出现 `Using filesort` 或 `Using temporary` 通常是坏消息。 `Using index` 是好消息。

`EXPLAIN ANALYZE`EXPLAIN ANALYZE 更进一步，它会**真实地执行**查询，并返回计划和**实际**的执行时间和行数。这是进行精确优化的终极武器。

我的练习记录 4.1：分析一个没有索引的查询

在 `orders` 表上， `order_date` 列没有索引。我们来查一下某一天有多少订单。

代码块

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM orders WHERE order_date = '2025-10-10';

```

我的（预估）结果：

代码块

```
                                                     QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=17000.00..17000.01 rows=1 width=8) (actual time=150.50..150.51 rows=1 loops=1)
   ->  Seq Scan on orders  (cost=0.00..16000.00 rows=400000 width=0) (actual time=0.02..120.10 rows=5000 loops=1)
         Filter: (order_date = '2025-10-10'::date)
         Rows Removed by Filter: 995000
 Planning Time: 0.15ms
 Execution Time: 151.00ms

```

我的解读：

1. `Seq Scan on orders` ： `Seq Scan` 就是**全表扫描**，这是最坏的情况。
2. `rows=400000` （预估）和 `actual ... rows=5000` （实际）：优化器预估要扫描40万行，实际返回了5000行。

3. `Execution Time: 151.00ms`：执行耗时 151 毫秒。

现在，我给 `order_date` 创建一个索引：

代码块

```
CREATE INDEX idx_orders_order_date ON orders(order_date);

```

再次执行 `EXPLAIN ANALYZE`：

代码块

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM orders WHERE order_date = '2025-10-10';

```

我的（预估）结果：

代码块

```
                                                        QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=125.00..125.01 rows=1 width=8) (actual time=5.50..5.51 rows=1 loops=1)
   ->  Index Only Scan using idx_orders_order_date on orders (cost=0.42..100.00 rows=5000 width=0) (actual time=0.05..3.10 rows=5000 loops=1)
         Index Cond: (order_date = '2025-10-10'::date)
 Planning Time: 0.25ms
 Execution Time: 6.00ms

```

我的解读：

1. `Index Only Scan`：访问类型变成了**索引扫描**，并且是 `Index Only`（只用索引就够了，不用回表），性能极好！
2. `rows=5000`：预估扫描行数大大减少。
3. `Execution Time: 6.00ms`：**执行时间从 151ms 降到了 6ms！** 效果立竿见影。

知识卡片：PostgreSQL 的索引万花筒

MySQL 的 InnoDB 主要使用 B-Tree 索引。而 PostgreSQL 提供了更多“特种兵”索引。

索引类型	核心原理	适用场景
B-Tree	平衡树结构，通用	最常用，支持 `=`, `>`, `<`, `>=`, `<=`, `BETWEEN` 等操作，适合高基数（唯一值多）的列。
Hash	哈希表	只支持 `=` 精确匹配。在 PG 10 之后才变得可靠。
GiST (Generalized Search Tree)	通用化搜索树	“框架”索引，可实现多种索引策略。常用于**地理空间数据 (PostGIS)**、全文搜索。
GIN (Generalized Inverted Index)	广义倒排索引	适合**多值类型**，如数组 `array`、`jsonb`、全文搜索。查询一个元素是否在数组中时，它比 B-Tree 快得多。
BRIN (Block Range Index)	块范围索引	存储每个数据块范围内的**元数据**（如最大/最小值）。适合与物理存储顺序**高度相关**的列，如时间序列数据。占空间极小。

我的注意点：

- **选择困难症？** 别怕，**90% 的场景下，B-Tree 都是你的最佳选择。**
- **新特性摘要：PostgreSQL 16 优化** PostgreSQL 16 对 B-Tree 索引的去重和压缩做了优化，使得索引体积更小，性能更好。同时，`EXPLAIN` 的输出也包含了更多关于 I/O 的详细信息。[18]

索引失效的常见“杀手”

1. **在索引列上使用函数或表达式：** `WHERE YEAR(order_date) = 2025` 会让 `order_date` 上的索引失效。应该改写为 `WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'`。
2. **`LIKE` 查询以通配符开头：** `WHERE user_name LIKE '%飞'` 无法使用索引。`WHERE user_name LIKE '李%'` 则可以。
3. **隐式类型转换：** 如果 `user_id` 是字符串类型，`WHERE user_id = 123` (数字) 可能会导致索引失效。
4. **`OR` 连接非索引列：** `WHERE indexed_col = 'A' OR unindexed_col = 'B'` 很可能导致全表扫描。
5. **数据分布不均：** 如果某个值的占比非常高（比如性别列），优化器可能认为走索引还不如全表扫描快。

第四周复盘

- **我的收获**：`EXPLAIN` 是我的新朋友，也是最重要的朋友。不看执行计划就写复杂 SQL，无异于“盲人摸象”。
- **遇到的困难**：`EXPLAIN` 的输出信息太多了，刚开始有点不知所措。我决定先重点关注 `type`，`key`，`rows`，`Extra` 这几项。
- **我的思考**：索引不是银弹。创建索引有成本（占用磁盘空间，拖慢写操作），维护索引也有成本。**创建正确的索引，和不创建错误的索引，同等重要**。下一周学习高级语法，我会时刻带着 `EXPLAIN` 去审视它们。

第五周：高级语法：窗口函数与 CTE

学习目标

1. **掌握窗口函数**：熟练使用 `ROW_NUMBER()`，`RANK()`，`DENSE_RANK()`，`LEAD()`，`LAG()` 等函数进行排名、跨行计算。
2. **理解 CTE**：学会使用 `WITH` 子句（公用表表达式）来拆解复杂查询，提高可读性。
3. **递归 CTE**：挑战高难度，用递归 CTE 处理层级数据（如组织架构）。
4. **探索 `LATERAL` JOIN**：了解 `LATERAL` 的特殊用法。
5. **数据透视**：学习 `PIVOT` 和 `UNPIVOT`（或用 `CASE WHEN` 模拟）。

知识卡片：窗口函数，SQL里的“上帝视角”

普通聚合函数会把多行压缩成一行。而**窗口函数**，会在保持原始行数不变的情况下，为每一行计算出一个“窗口”内的聚合值。

代码块

```
<窗口函数>() OVER (
    [PARTITION BY <分组列>]
    [ORDER BY <排序列>]
    [ROWS/RANGE BETWEEN <窗口范围>]
)

```

- `PARTITION BY`：定义“窗口”的边界，类似于 `GROUP BY`。
- `ORDER BY`：决定窗口内如何排序。
- `ROWS/RANGE BETWEEN`：定义窗口具体的计算范围（如“当前行和前面2行”）。

常用窗口函数	分类	作用

`ROW_NUMBER()`	排名函数	连续排名 (1, 2, 3, 4)
`RANK()`	排名函数	并列跳跃排名 (1, 2, 2, 4)
`DENSE_RANK()`	排名函数	并列连续排名 (1, 2, 2, 3)
`LEAD()`, `LAG()`	偏移函数	获取上/下 N 行的值（计算同比、环比）
`SUM()`, `AVG()`, `COUNT()`	聚合函数	计算窗口内的聚合值（如“移动平均”）

我的练习记录 5.1：计算每个部门工资最高的前3名员工

代码块

```
SELECT
    *
FROM (
    SELECT
        employee_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as rn
    FROM
        employees
) AS ranked_employees
WHERE
    rn <= 3;

```

我的复盘：

- 窗口函数不能直接用在 `WHERE` 子句中，所以需要一层子查询或 CTE。
- `PARTITION BY department` 把数据按部门分成了不同的“窗口”。
- `ORDER BY salary DESC` 在每个窗口内按工资降序排列。
- `ROW_NUMBER()` 为每个窗口内的行分配一个从 1 开始的连续排名。

知识卡片：CTE，让我的 SQL 不再是一团乱麻

CTE (Common Table Expression)，公用表表达式，使用 `WITH` 关键字定义。它就像一个临时的、只在当前查询中有效的“视图”。

好处：

1. **可读性**：把一个大查询拆分成多个逻辑清晰的小步骤。

2. **可维护性**：修改逻辑时，只需要修改对应的 CTE 即可。
3. **可复用性**：一个 CTE 可以在后续查询中被多次引用。
4. **支持递归**：这是子查询做不到的。

我的练习记录 5.2：用 CTE 改写练习 5.1

代码块

```
WITH ranked_employees AS (
    -- 第一步：计算排名
    SELECT
        employee_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as rn
    FROM
        employees
)
-- 第二步：从排名结果中筛选
SELECT
    employee_name,
    department,
    salary
FROM
    ranked_employees
WHERE
    rn <= 3;

```

我的复盘：

- 逻辑是不是清晰多了？第一步专门负责排名，第二步负责筛选。
- 对于非常复杂的报表，可能会有 5-6 个甚至更多的 CTE 串联起来，像流水线一样处理数据。

挑战：递归 CTE 查询组织架构

假设 `employees` 表有 `employee_id` 和 `manager_id` 两列。我想查询某个员工（比如 ID=1）的所有下属。

代码块

```
WITH RECURSIVE subordinates AS (
    -- 初始部分 (Anchor Member)
    SELECT
        employee_id,
        employee_name,
```

```
        manager_id
    FROM
        employees
    WHERE
        employee_id = 1 -- 从老板开始

    UNION ALL

    -- 递归部分 (Recursive Member)
    SELECT
        e.employee_id,
        e.employee_name,
        e.manager_id
    FROM
        employees e
    JOIN
        subordinates s ON e.manager_id = s.employee_id -- 连接条件是关键
)
SELECT * FROM subordinates;

```

我的复盘：

- 递归 CTE 必须包含两部分：一个不引用自己的初始 `SELECT`，和一个引用自己的递归 `SELECT`，用 `UNION ALL` 连接。
- `JOIN subordinates s ON e.manager_id = s.employee_id` 这句是核心：在员工表里找到那些“经理 ID 等于当前下属集合中员工 ID”的人，把他们也加入到下属集合中，然后循环往复。

新特性摘要：SQL:2023 与图查询我查到的资料显示，SQL:2023 标准正式引入了属性图查询（Property Graph Queries, PGQ）功能。[1] 这意味着未来可以用更原生、更直观的语法来处理类似组织架构、社交网络这样的图数据，而不再完全依赖递归 CTE。例如，可能会有类似 `MATCH (a)-[is_manager_of]->(b)` 这样的语法。Oracle 23ai 已经率先支持了这一特性。[61]

第五周复盘

- **我的巨大收获**：窗口函数和 CTE 共同为我打开了高级数据分析的大门。以前觉得遥不可及的“分组排名”、“移动平均”、“用户分层”等分析，现在我有了清晰的实现思路。
- **`LATERAL` JOIN 初探**：我了解到 `LATERAL` 允许右侧的子查询引用左侧表的列，这对于某些复杂的“top-N”查询非常有用，可以替代一些窗口函数的写法。例如，查询每个部门工资最高的员工。我计划在之后的练习中深入使用。
- **我的思考**：工具的进化是为了更优雅地解决问题。从子查询到 CTE，再到未来原生的图查询，SQL 语言本身也在不断演进，让复杂逻辑的表达越来越清晰。作为学习者，既要掌握“经典款”（如递归 CTE），也要关注“新款”（如 PGQ），才能保持竞争力。

第六周：数据建模与项目实践（一）：电商分析

学习目标

1. **理解建模流程**：了解从业务需求到概念模型、逻辑模型、再到物理模型的全过程。
2. **掌握 ER 图**：能读懂 ER 图，并能根据业务场景设计简单的 ER 图。
3. **物理设计**：将逻辑模型转化为具体的、带有数据类型和约束的 `CREATE TABLE` 语句。
4. **数据初始化**：编写 `INSERT` 脚本，为数据库填充初始的模拟数据。
5. **业务 KPI 查询**：基于建好的模型，编写 SQL 查询典型的电商业务指标（如日活用户、转化率、客单价等）。

从需求到模型：我的第一个电商项目

业务需求：我想搭建一个最简化的电商系统，需要管理**用户**、**商品**和**订单**。

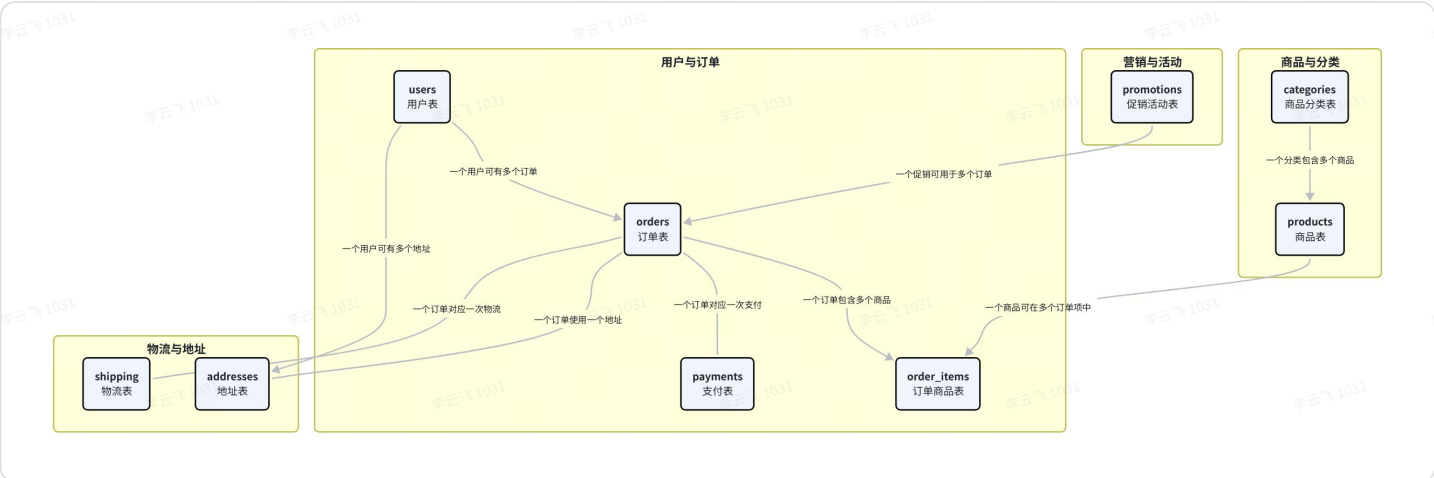
- 用户可以注册登录。
- 商品有名称、价格、库存。
- 用户可以创建订单，一个订单可以包含多种商品，每种商品有购买数量。

第一步：概念模型识别核心实体（Entities）：**用户 (User)**、**商品 (Product)**、**订单 (Order)**。

第二步：逻辑模型（ER 图）设计实体之间的关系（Relationships）：

- 一个**用户**可以有多个**订单**（一对多）。
- 一个**订单**包含多种**商品**，一个**商品**也可能出现在多个**订单**中（多对多）。
- “多对多”关系需要一个**中间表（关联表）**来承载，我称之为 `order_items`。

这就是我上周已经画好的 ER 图：



第三步：物理模型（`CREATE TABLE` 脚本）

我将使用 PostgreSQL 语法来编写。

代码块

```sql
-- 用户表 (users)
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY, -- SERIAL 自增主键
    user_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL, -- 邮箱唯一
    password_hash CHAR(64) NOT NULL, -- 存储密码哈希值
    registration_date TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP, -- 带时区的时间戳，默认为当前时间
    city VARCHAR(50)
);

-- 商品表 (products)
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL CHECK (price > 0), -- 价格必须大于0
    stock INT NOT NULL CHECK (stock >= 0)
);

-- 订单表 (orders)
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    order_date TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10, 2),
    status VARCHAR(20) DEFAULT 'pending',
    FOREIGN KEY (user_id) REFERENCES users(user_id) -- 外键约束
);

-- 订单商品关联表 (order_items)
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    price_per_item DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    UNIQUE (order_id, product_id) -- 同一订单中，一个商品只能出现一次
);

```

我的设计思考：

- **数据类型选择**：`SERIAL` 方便地创建了自增主键。`TIMESTAMPTZ` (timestamp with time zone) 是处理时间的最佳实践，可以避免时区问题。`DECIMAL` 用于存储金额，避免浮点数精度问题。
- **约束**：我用了 `PRIMARY KEY`，`NOT NULL`，`UNIQUE`，`CHECK`，`FOREIGN KEY` 五种约束来保证数据的**完整性**和**一致性**。外键是维护表间关系的核心。

数据初始化

我需要写一些 `INSERT` 语句来填充一些模拟数据，以便后续进行查询。

代码块

```
-- 插入用户
INSERT INTO users (user_name, email, password_hash, city) VALUES
('李云飞', 'liyunfei.07@example.com', 'some_hash_value', '北京'),
('张三', 'zhangsan@example.com', 'another_hash_value', '上海');

-- 插入商品
INSERT INTO products (product_name, price, stock) VALUES
('SQL学习指南', 99.99, 100),
('高性能服务器', 4999.00, 10);

-- 模拟一次下单流程（这在应用中会是多个步骤，并包含在事务里）
-- 1. 创建订单
INSERT INTO orders (user_id, status) VALUES (1, 'completed') RETURNING order_id;
-- 假设返回的 order_id 是 101

-- 2. 插入订单项
INSERT INTO order_items (order_id, product_id, quantity, price_per_item) VALUES
(101, 1, 2, 99.99),
(101, 2, 1, 4999.00);

-- 3. 更新订单总金额
UPDATE orders
SET total_amount = (SELECT SUM(quantity * price_per_item) FROM order_items WHERE order_id = 101)
WHERE order_id = 101;

```

我的业务 KPI 查询练习

1. 日活跃用户 (DAU) 查询假设有一个 `user_login_log` 表，记录用户登录。

代码块

```
SELECT
    COUNT(DISTINCT user_id) AS dau
```

```
FROM
    user_login_log
WHERE
    login_time >= '2025-10-10 00:00:00' AND login_time < '2025-10-11 00:00:00';

```

我的注意点： `COUNT(DISTINCT ...)` 是去重计数，防止一个用户当天登录多次被重复计算。

2. 用户月留存率查询 9 月份注册的用户，在 10 月份有多少人还活跃（登录过）。

代码块

```
WITH september_users AS (
    SELECT DISTINCT user_id
    FROM users
    WHERE registration_date >= '2025-09-01' AND registration_date < '2025-10-01'
),
october_active_users AS (
    SELECT DISTINCT user_id
    FROM user_login_log
    WHERE login_time >= '2025-10-01' AND login_time < '2025-11-01'
)
SELECT
    (COUNT(t2.user_id)::DECIMAL / COUNT(t1.user_id)) * 100 AS retention_rate
FROM
    september_users t1
LEFT JOIN
    october_active_users t2 ON t1.user_id = t2.user_id;

```

我的复盘： 这是一个典型的 CTE 应用。先把两组用户（9月注册，10月活跃）分别定义出来，再进行关联计算。`LEFT JOIN` 保证了所有 9 月用户都被统计在分母中。

3. 客单价 (Average Order Value, AOV)

代码块

```
SELECT
    AVG(total_amount) AS aov
FROM
    orders
WHERE
    status = 'completed' AND order_date >= '2025-10-01' AND order_date < '2025-11-01';

```

第六周复盘

- **从抽象到具体**：这一周的学习让我体验了从一个模糊的“想法”到一个结构清晰、约束严格的数据库的完整过程。这是我第一次感受到“设计”的力量。
- **SQL 是活的**：我写的不再是孤立的查询，而是一个个服务于业务目标的“数据服务”。留存率、转化率这些以前只在 PPT 里听到的词，现在我能亲手把它们从数据里“挖”出来。
- **我的不足**：目前的模型还非常简单。没有考虑商品分类、用户地址、优惠券、支付记录等等。这让我意识到，真实世界的建模远比这复杂。但我已经迈出了第一步，并且理解了建模的基本思想：**识别实体、定义关系、明确属性、施加约束**。
- **下一步**：有了数据和模型，下一步就是如何让它跑得更快、更稳。我已经迫不及待要进入后面更深入的优化和工程化主题了。

第七周：玩转半结构化与特殊数据

学习目标

1. **JSON 数据处理**：掌握在 PostgreSQL 和 MySQL 中查询 `JSON` / `JSONB` 类型数据的核心操作符和函数。
2. **JSON 实战**：能从存储为 JSON 的日志数据中提取字段，进行过滤和聚合。
3. **其他数据类型初探**：了解PostgreSQL 强大的扩展能力，特别是 PostGIS 在处理地理空间数据方面的应用。
4. **跨库语法差异**：对比不同数据库在处理半结构化数据时的语法差异。

知识卡片：为什么要在关系数据库里用 JSON?

我一开始有个疑问：关系数据库不是强调结构化吗，为什么还要引入 JSON 这种“非结构化”的东西?

> 💭 **我的理解：这是对现实世界的妥协与拥抱。**
>
> 1. **灵活性**：不是所有数据都适合预先定义好严格的表结构。比如，用户自定义标签、商品的规格参数，这些字段可能千变万化。用 JSON 存储，可以随时增加新属性，而无需 `ALTER TABLE`。
> 2. **外部数据**：现代应用大量与第三方 API 交互，返回的数据通常就是 JSON 格式。直接将其存入数据库，可以避免复杂的解析和映射过程，简化开发。
> 3. **日志记录**：应用日志通常是半结构化的，包含固定的字段（如时间、级别）和不定的上下文信息（如错误堆栈、请求参数）。用 JSON 存储日志正合适。

`JSON` vs `JSONB` in PostgreSQL

- `JSON`：以**文本**形式存储，保留原始格式（包括空格和key的顺序）。插入快，查询慢。
- `JSONB`：以**二进制**形式存储，会去除多余空格和key的顺序，并对key进行去重。插入稍慢（需要解析），但查询**快得多**，并且**支持索引**。
- **我的选择：无脑选** `JSONB`，除非有特殊需求需要保留原始 JSON 文本的完整性。

PostgreSQL `JSONB` 查询实践

PostgreSQL 提供了非常丰富的操作符来“解剖” `JSONB`。

操作符	作用	示例
`->`	按 Key 获取 `JSON` 对象字段（返回 `jsonb`）	`payload -> 'user'`
`->>`	按 Key 获取 `JSON` 对象字段（返回 `text`）	`payload ->> 'user_name'`
`->#`	按路径获取 `JSON` 对象字段（返回 `jsonb`）	`payload ->#` `'{user,address,city}'`
`->>#`	按路径获取 `JSON` 对象字段（返回 `text`）	`payload ->>#` `'{user,address,city}'`
`?`	判断顶层 Key 是否存在	`payload ? 'error_code'`
`@>`	判断左边的 `JSONB` 是否**包含**右边的 `JSONB`	`payload @> '{"level":` `"error"}'`

我的练习记录 7.1：查询 JSON 格式的日志

假设我有一张 `app_logs` 表，其中 `log_payload` 是 `jsonb` 类型。

代码块

```
CREATE TABLE app_logs (
    id SERIAL PRIMARY KEY,
    log_time TIMESTAMPTZ,
    log_payload JSONB
);

INSERT INTO app_logs (log_time, log_payload) VALUES
('2025-11-01 10:00:00', '{"level": "info", "message": "User logged in", "user_id": 123, "ip": "192.168.1.1"}'),
('2025-11-01 10:05:00', '{"level": "error", "message": "Failed to connect to payment gateway", "error_code": 5003, "request_id": "xyz-abc"}');
```

```
10
```

查询需求1：找到所有 `level` 为 `error` 的日志。

代码块

```
1  -- 使用 @> 操作符，因为它能利用 GIN 索引，性能最好
2  SELECT
3      log_time,
4      log_payload ->> 'message' AS error_message
5  FROM
6      app_logs
7  WHERE
8      log_payload @> '{"level": "error"}';
9
```

查询需求2：统计每个 `user_id` 的登录次数。

代码块

```
1  SELECT
2      log_payload ->> 'user_id' AS user_id,
3      COUNT(*) AS login_count
4  FROM
5      app_logs
6  WHERE
7      log_payload ->> 'message' = 'User logged in' AND (log_payload ->>
   'user_id') IS NOT NULL
8  GROUP BY
9      user_id;
10
```

我的复盘：

- `->>` 返回 `text`，可以直接用于 `WHERE` 条件的比较、`GROUP BY` 和 `ORDER BY`。
- 对于 `JSONB` 的查询，创建 **GIN 索引**至关重要。例如 `CREATE INDEX idx_logs_payload_gin ON app_logs USING GIN (log_payload);`。有了这个索引，`@>` 操作符的查询会快如闪电。

跨库语法对比：JSON 处理

数据库	提取值	语法示例	索引支持

PostgreSQL	`->>`，`->` 等操作符	`payload ->> 'user.name'` (需用 `->#`)	**GIN 索引** (非常高效)
MySQL (5.7+)	`JSON_EXTRACT`，`->` (简写)	`JSON_EXTRACT(payload, '$.user.name')` 或 `payload->"$.user.name"`	5.7 支持在生成列上建索引，8.0+ 支持多值索引
SQL Server (2016+)	`JSON_VALUE`，`JSON_QUERY`	`JSON_VALUE(payload, '$.user.name')`	支持在计算列上建索引
Oracle	`JSON_VALUE`，`.` (简写)	`log.payload.user.name`	`JSON_SEARCH` 索引
Snowflake	`:`	`payload:user.name`	自动优化，无需手动建索引

我的注意点：虽然各家都在支持 JSON，但语法差异巨大。特别是路径表达式，PostgreSQL 用 `{}`，而 MySQL、SQL Server 遵循 `$.` 的 `JSONPath` 语法。Snowflake 的语法最为简洁。进行跨库迁移时，JSON 处理是重点改造对象。**Oracle 23ai** 推出的 **JSON Relational Duality** 视图，更是让 JSON 数据可以直接像关系表一样操作，这可能是未来的一个重要方向。[61]

地理空间数据：PostGIS 初探

PostGIS 是 PostgreSQL 的一个强大扩展，让 PG 变成了事实上的地理空间数据库标准。

代码块

```
-- 启用 PostGIS 扩展
CREATE EXTENSION postgis;

-- 创建带地理信息点的表
CREATE TABLE restaurants (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    geom GEOMETRY(Point, 4326) -- 存储 WGS84 坐标系的点
);

-- 插入一个餐馆
INSERT INTO restaurants (name, geom) VALUES
('My Favorite Cafe', ST_SetSRID(ST_MakePoint(116.404, 39.915), 4326)); -- 北京市中心的某个点

-- 查询离我最近的5家餐馆
SELECT
    name,
```

```sql
    ST_Distance(geom, ST_SetSRID(ST_MakePoint(116.405, 39.916), 4326)) AS distance
FROM
    restaurants
ORDER BY
    distance
LIMIT 5;

```

我的复盘：这太酷了！只需要 `ST_Distance` 这样的函数，就能进行复杂的地理空间计算。背后依赖的是 **GiST 索引**，它能高效地处理二维、三维空间数据。

第七周复盘

- **思维转变**：我不再认为数据库只能存储规整的表格数据。对于那些“长得不太规则”的数据，SQL 同样有强大的武器库来对付它们。
- **PostgreSQL 的魅力**：我深刻感受到了 PG “开源 Oracle” 的称号名不虚传。其强大的 `JSONB` 支持和 `PostGIS` 这样的杀手级扩展，让它在处理复杂数据类型时游刃有余。
- **我的下一个目标**：理解了如何处理更多样的数据，我需要建立一个更宏观的视角，去理解不同类型的数据库系统（OLTP 和 OLAP）是如何在整个数据架构中分工协作的。

第八周：OLAP 与数仓的世界观

学习目标

1. **区分 OLTP 和 OLAP**：清晰地描述两种系统的设计目标、工作负载和技术特点。
2. **理解维度建模**：掌握数据仓库建模的核心思想，特别是星型模型和雪花模型。
3. **核心概念**：能解释什么是事实（Fact）、维度（Dimension）、SCD（缓慢变化维度）。
4. **动手设计**：为一个简单的业务场景设计一个星型模型。

知识卡片：OLTP vs. OLAP，数据库世界的“双子星”

💡 **OLTP (Online Transaction Processing)**
在线事务处理

- **核心目标**：**处理**日常的、高并发的**事务**。
- **我的比喻**：银行的**柜台**。负责快速、准确地处理每一笔存取款业务。

💡 **OLAP (Online Analytical Processing)**
在线分析处理

- **核心目标**：支持复杂的**分析**查询，进行决策支持。

- **设计重点**：高并发、低延迟、数据一致性、面向事务。
- **数据模型**：高度规范化（三范式），避免数据冗余。
- **数据库代表**：MySQL, PostgreSQL, Oracle (用于其核心事务功能)。

- **我的比喻**：银行的**分析部**。负责回顾过去一整年的交易数据，分析客户行为，制定营销策略。
- **设计重点**：查询性能、数据聚合、面向分析。
- **数据模型**：反规范化，允许冗余以提高查询速度（维度建模）。
- **数据库代表**：ClickHouse, Snowflake, BigQuery, Redshift。

维度建模：为分析而生的艺术

维度建模是数据仓库的基石，由 Ralph Kimball 大师提出。其核心思想是把数据分为**事实**和**维度**。

- **事实 (Fact)**：通常是**数字**、**度量值**，是我们想要分析的核心指标。比如，订单的“销售额”、“数量”。事实表通常行数非常多。
- **维度 (Dimension)**：我们观察事实的**角度**和**上下文**。比如，“时间”、“地点”、“用户”、“商品”。维度提供了分析的切入点，如“按地区分析销售额”、“按用户性别分析购买偏好”。

星型模型 (Star Schema)这是最经典的维度模型。它由一个中心的**事实表**和周围环绕的多个**维度表**组成，就像一颗星星。

我的练习记录 8.1：设计一个电商销售分析的星型模型

业务需求：从时间、用户、商品、地区等多个维度分析销售情况。

- **事实**：销售额、销售数量。
- **维度**：时间、用户、商品、地区。

模型设计：

1. **事实表** `fact_sales`：
   - `date_key` (外键, 关联时间维度)
   - `user_key` (外键, 关联用户维度)
   - `product_key` (外键, 关联商品维度)
   - `location_key` (外键, 关联地区维度)
   - `sales_amount` (度量)
   - `sales_quantity` (度量)
2. **维度表** `dim_date`：
   - `date_key` (主键)
   - `full_date`

- `year`, `quarter`, `month`, `day`, `day_of_week` ...

3. **维度表 `dim_user`**：
   - `user_key` (主键)
   - `user_id` (源系统 ID)
   - `user_name`, `gender`, `age_group`, `registration_date` ...
4. **维度表 `dim_product`**：
   - `product_key` (主键)
   - `product_id` (源系统 ID)
   - `product_name`, `category`, `brand` ...
5. **维度表 `dim_location`**：
   - `location_key` (主键)
   - `country`, `province`, `city` ...

查询示例：查询 2025 年第三季度，北京地区，各商品品类的总销售额

代码块

```sql
SELECT
    p.category,
    SUM(f.sales_amount) AS total_sales
FROM
    fact_sales f
JOIN
    dim_date d ON f.date_key = d.date_key
JOIN
    dim_location l ON f.location_key = l.location_key
JOIN
    dim_product p ON f.product_key = p.product_key
WHERE
    d.year = 2025 AND d.quarter = 3 AND l.city = '北京'
GROUP BY
    p.category;

```

我的复盘：

- **反规范化**：`dim_date` 表中存在大量冗余（比如每天都存一遍年份），但这极大地简化了查询。我不需要再用 `YEAR()` 函数，直接 `WHERE d.year = 2025` 即可，查询效率更高。
- **查询友好**：星型模型的查询逻辑非常直观，就是事实表和一堆维度表的 JOIN，GROUP BY 维度属性，SUM 事实度量。

雪花模型 (Snowflake Schema)如果把星型模型中的某个维度表（比如商品维度）进一步规范化，拆分成更小的表（如商品表和品类表），就形成了雪花模型。

- **优点**：节省存储空间。
- **缺点**：查询时需要更多的 JOIN，可能影响性能。
- **我的选择**：在存储成本不再是首要矛盾的今天，**优先使用星型模型**，它的查询性能和模型简洁性通常更具优势。

缓慢变化维度 (SCD - Slowly Changing Dimension)

维度的属性不是一成不变的。比如，用户可能会搬家（城市变了），商品可能会调整品类。如何处理这些变化，就是 SCD 要解决的问题。

SCD 类型	处理方式	我的理解
SCD1	**直接覆盖**	不保留历史。用户搬家了，直接更新地址。简单粗暴，但会丢失历史信息。
SCD2	**增加新行** + 版本控制	保留所有历史。用户搬家了，把旧地址记录标记为“过期”，并插入一条新地址记录标记为“当前”。这是数仓中最常用的方式。
SCD3	增加“之前”的列	只保留上一次的历史。比如增加 `previous_city` 列。适用场景有限。

第八周复盘

- **两种世界观**：我彻底理解了，不能用设计 OLTP 系统的思路来设计 OLAP 系统。前者追求的是操作的原子性和数据的规范性，后者追求的是查询的灵活性和性能的极致。
- **维度建模是核心**：我意识到，数据分析的质量，很大程度上取决于数据仓库模型设计的好坏。一个好的模型，能让复杂的分析查询变得简单、高效。
- **我的新大陆**：OLAP 和数据仓库为我打开了一个全新的世界。我开始思考，那些 TB、PB 级别的海量数据，是如何被现代数据引擎高效处理的。下一周，我将去探索这些“巨兽”们的秘密。

第九周：云数仓与新一代引擎巡礼

学习目标

1. **了解云数仓三巨头**：对 Snowflake, Google BigQuery, Amazon Redshift 的核心架构和特点有一个基本认知。
2. **认识新锐力量**：了解 ClickHouse 和 DuckDB 这两个备受瞩目的开源分析引擎。
3. **理解存算分离**：明白为什么“存算分离”是现代云数仓的标配架构。
4. **关注最新趋势**：通过查阅资料，了解这些平台在 2025-2026 年的最新动态，特别是与 AI 的结合。

知识卡片：云数仓的核心革命——存算分离

传统的一体化数据库架构，计算资源（CPU）和存储资源（磁盘）是紧密耦合的。扩容时必须两者一起扩，缺乏弹性。

存算分离 (Separation of Compute and Storage) 架构，则将两者解耦：

- **存储层**：使用廉价、可无限扩展的对象存储（如 AWS S3, Google Cloud Storage）。
- **计算层**：由多个无状态的计算集群（或虚拟仓库）组成。

> ⭐ **存算分离带来的巨大优势：**
>
> 1. **极致弹性**：存储和计算可以**独立**扩缩容。数据量大，就增加存储；查询任务多、计算复杂，就增加计算集群。用完即停，按需付费，成本效益极高。
> 2. **多集群负载隔离**：可以为不同的任务（如数据ETL、BI报表、AI训练）分配独立的计算集群，它们共享同一份数据，但计算上互不干扰。
> 3. **数据共享**：数据只需存一份，就可以被多个应用、多个团队安全地共享访问。

云数仓三巨头：Snowflake, BigQuery, Redshift

平台	核心特点	2025-2026 最新动态与我的关注点
Snowflake	存算分离的典范，多集群虚拟仓库，跨云数据共享 (Data Sharing)，Snowpark (用 Python/Java 等直接操作数据)。	**AI 战略是重中之重**。推出 **Snowflake Cortex AI**，让用户可以直接用 SQL 调用 LLM 进行情感分析、文本摘要等。收购了 Neeva，增强搜索能力。**Iceberg 表**成为其数据湖战略的核心。[48]
Google BigQuery	完全 Serverless，Google 强大基础设施的体现。通过 `slots` 管理计算资源。与 Google Cloud 生态（AI Platform, Looker）深度集成。	**与 Gemini 深度集成**，提供自然语言查询、代码辅助等功能。**BigQuery Studio** 统一了数据工程、分析和科学的开发体验。同样在积极拥抱 **Apache Iceberg**。[51]
Amazon Redshift		

	最早的云数仓之一，从基于 PostgreSQL 的 MPP 架构演进而来。近年来也在向存算分离（Redshift Spectrum, RA3 节点）演进。	**Redshift Serverless** 持续优化，降低使用门槛。**AI 驱动的优化**，如自动物化视图、自动表优化。与 AWS SageMaker（机器学习平台）集成更紧密。

我的思考：一个明显的趋势是，三大云数仓都在**全面拥抱 AI/LLM**，试图让 SQL 不再是数据交互的唯一语言，同时也用 AI 来赋能 SQL 本身的开发和优化。另一个趋势是，**Apache Iceberg** 正在成为云上数据湖的事实标准，让这些数仓能更好地读写开放格式的数据。

新锐力量：ClickHouse 与 DuckDB

如果说云数仓是“航母”，那这两个就是“快艇”和“瑞士军刀”。

💡 **ClickHouse**

- **定位：极致性能**的开源列式数据库。为 OLAP 而生。
- **核心技术**：列式存储、向量化执行、数据压缩。
- **我的印象：快，非常快！** 在某些分析场景下，比通用数据库快几个数量级。特别适合海量日志分析、用户行为分析、监控指标存储等。
- **最新动态**：ClickHouse Cloud 发展迅速，提供了存算分离的云版本。社区也在不断增强其 `JOIN` 能力和对标准 SQL 的兼容性。

💡 **DuckDB**

- **定位：进程内 (In-Process)** 的分析型数据库。
- **核心技术**：向量化执行、免安装、直接读写 Parquet/CSV 文件。
- **我的印象：数据科学家的瑞士军刀**。它不是一个需要部署的服务，而是一个可以被 Python, R, Java 等语言直接调用的**库**。可以直接在笔记本电脑上对几个 G 的 Parquet 文件进行飞快的 SQL 分析，无需导入任何数据库。
- **最新动态**：生态系统蓬勃发展，与 Pandas、Arrow 等数据科学生态无缝集成。正在探索并行和分布式执行能力。

分布式 SQL 引擎：TiDB, CockroachDB, Trino

- **TiDB / CockroachDB**：被称为 **NewSQL** 或 **HTAP** (Hybrid Transactional/Analytical Processing)。它们的目标是**同时支持**高并发的 OLTP 负载和复杂的 OLAP 查询，并提供水平扩展能力。它们通常与 MySQL (TiDB) 或 PostgreSQL (CockroachDB) 兼容。
- **Trino (原 PrestoSQL)**：一个**联邦查询引擎**。它自己不存储数据，但可以像一个“章鱼”一样，同时连接并查询多种不同的数据源（如 MySQL, Hive, Kafka），并提供统一的 SQL 接口。

第九周复盘

- **视野的震撼**：这一周，我看到了数据处理技术的“星辰大海”。从庞大的云数仓，到小而美的DuckDB，不同的工具服务于不同的场景，但都在追求极致的性能和弹性。
- **“列存”与“向量化”**：我反复看到这两个词。我理解到，**列式存储**通过只读取查询所需的列，极大地减少了 I/O。而**向量化执行**则是一种 CPU 友好的计算方式，一次处理一批数据，而不是一行一行地处理。这是现代分析引擎速度飞快的两大秘诀。
- **我的技术选型观**：没有最好的数据库，只有最合适的场景。我现在有了一个初步的选型框架：
  - **高并发在线业务**：MySQL, PostgreSQL。
  - **企业级大规模数据分析**：Snowflake, BigQuery。
  - **海量日志、监控数据实时分析**：ClickHouse。
  - **本地数据探索、数据科学**：DuckDB。
  - **需要同时支持事务和分析的复杂场景**：TiDB。
- **下一步**：了解了这么多强大的“武器”，我需要学习如何将它们应用在实际的“工程”中，保证数据的质量、安全和可靠。

第十周：工程与可靠性：从 SQL 到数据工程

学习目标

1. **理解幂等性**：学会编写“可以安全地重复执行”的 SQL 脚本，特别是 `INSERT ... ON CONFLICT` (UPSERT)。
2. **数据质量入门**：了解保证数据质量的基本方法，如约束、数据探查查询。
3. **流式 SQL 初探**：对 Flink SQL 等流计算引擎有一个初步的印象，理解流处理和批处理的区别。
4. **安全与合规**：了解行级安全（Row-Level Security）等数据库安全特性。

知识卡片：幂等性，数据工程师的“安全带”

幂等性（Idempotence）是一个非常重要的工程概念，指的是一个操作执行一次和执行多次，产生的结果是完全相同的。

为什么需要幂等性？在数据处理中，任务失败重试是常态（网络抖动、机器故障等）。如果一个 `INSERT` 操作不幂等，重试时就会插入重复数据。

SQL 中的幂等实现：UPSERTUPSERT = UPDATE + INSERT。如果记录存在，就更新；如果不存在，就插入。

数据库	UPSERT 语法
PostgreSQL	`INSERT INTO ... ON CONFLICT (unique_column) DO UPDATE SET ...`

MySQL	`INSERT INTO ... ON DUPLICATE KEY UPDATE ...`
SQL Server	`MERGE INTO ...`
Oracle	`MERGE INTO ...`
SQLite	`INSERT INTO ... ON CONFLICT(unique_column) DO UPDATE SET ...`

我的练习记录 10.1：用 UPSERT 更新商品库存

假设我要从一个外部源同步商品库存数据到我的 `products` 表。

代码块

```sql
-- PostgreSQL 写法
INSERT INTO products (product_id, product_name, stock)
VALUES (101, '新商品A', 50)
ON CONFLICT (product_id) DO UPDATE SET
    stock = EXCLUDED.stock, -- EXCLUDED 是一个虚拟表，代表试图插入的数据
    product_name = EXCLUDED.product_name;

```

我的复盘：有了这个 `ON CONFLICT` 子句，无论我执行这个脚本多少次，`product_id` 为 101 的商品最终库存都会是 50，而且不会报错，也不会产生重复数据。这对于编写可靠的数据同步脚本至关重要。

数据质量：垃圾进，垃圾出 (Garbage In, Garbage Out)

数据质量是数据分析和机器学习的生命线。保证数据质量的方法有很多：

1. **数据库约束**：`NOT NULL`，`UNIQUE`，`CHECK`，`FOREIGN KEY` 是第一道防线。在第六周的建模中我已经用到了。
2. **数据探查/剖析 (Data Profiling)**：编写 SQL 查询来“审视”数据，检查是否存在异常。
   - `SELECT COUNT(*) FROM my_table WHERE some_column IS NULL;` -- 检查空值
   - `SELECT some_column, COUNT(*) FROM my_table GROUP BY some_column HAVING COUNT(*) > 1;` -- 检查唯一列是否有重复
   - `SELECT MIN(date_column), MAX(date_column) FROM my_table;` -- 检查时间范围是否符合预期
3. **数据测试**：使用 dbt, Great Expectations 等工具，将数据质量的断言自动化，集成到数据处理流程中。

知识卡片：批处理 vs. 流处理

💡 **批处理 (Batch Processing)**

- **处理方式**：处理**有界**的数据集，一次处理一大批。
- **我的比喻**：**洗衣服**。攒满一筐衣服，然后一次性扔进洗衣机洗。
- **延迟**：高。通常是小时级、天级。
- **场景**：T+1 的数据仓库报表、离线机器学习训练。
- **工具**：Hive, Spark, MapReduce。

💡 **流处理 (Stream Processing)**

- **处理方式**：处理**无界**的数据流，来一条处理一条。
- **我的比喻**：**自来水龙头**。水流持续不断，处理也是实时发生的。
- **延迟**：低。通常是毫秒级、秒级。
- **场景**：实时风控、实时推荐、实时监控大盘。
- **工具**：**Flink**, Spark Streaming, Storm。

Flink SQL：用 SQL 写流处理程序 Flink 是流处理领域的王者。Flink SQL 允许我们用熟悉的 SQL 语法来定义流数据的处理逻辑，极大地降低了流处理的门槛。

新特性摘要：Apache Flink 2.0.0 之后的演进我查到的资料显示，Flink 2.0.0 之后的一个重要方向是**流批一体**的真正落地。[30] 这意味着，开发者可以用同一套 API 和 SQL 来处理流数据和批数据。另一个重点是**物化表（Materialized Table）**和**动态表（Dynamic Table）**的概念，它们让流式聚合和 JOIN 的表达更加自然。

安全与合规：行级安全 (RLS)

行级安全（Row-Level Security）是一种精细的访问控制机制，它能让不同的用户在查询**同一张表**时，看到**不同的行**。

应用场景：

- 在一个多租户 SaaS 应用中，A 公司的用户只能看到 A 公司的数据。
- 在一个销售管理系统中，销售经理只能看到自己团队成员的客户数据。

在 PostgreSQL 中，可以通过 `CREATE POLICY` 来实现。

代码块

```
-- 假设 session 中有当前用户的公司 ID: current_setting('app.company_id')
CREATE POLICY company_isolation_policy ON orders
FOR SELECT
USING (company_id = current_setting('app.company_id')::int);

-- 启用 RLS
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

```

我的复盘：RLS 把数据权限的控制下沉到了数据库层面，而不是在应用层做各种 `WHERE` 条件的拼接。这更安全、更可靠，也更容易维护。

第十周复盘

- **从“怎么写”到“怎么写得好”**：这一周我的关注点发生了转变。我不再只关心 SQL 能不能跑通，而是关心它是否健壮（幂等性）、是否可靠（数据质量）、是否安全（RLS）。
- **数据工程师的视角**：我开始理解数据工程师日常工作的一部分——确保数据流的稳定、准确和安全。SQL 在其中扮演了核心角色。
- **流处理的大门**：Flink SQL 让我非常兴奋。能够用 SQL 来处理实时数据流，这意味着 SQL 的能力边界被极大地拓宽了。我计划在未来深入学习它。

第十一周：项目实践（二）：日志告警系统

学习目标

1. **时序数据处理**：练习在 SQL 中处理时间序列数据。
2. **复杂窗口函数应用**：综合运用窗口函数解决“连续 N 次”这类问题。
3. **性能优化实战**：为一个高频写入、高频查询的场景设计表结构和索引。
4. **物化视图**：了解并使用物化视图来加速复杂聚合查询。

项目需求：网站错误率实时告警

业务需求：监控网站的 API 访问日志，如果任何一个 API 端点在**连续 5 分钟内**，每分钟的**错误率（HTTP 状态码 >= 500）都超过 5%**，就触发告警。

第一步：表设计

我需要一张表来存储 API 访问日志。考虑到写入量可能非常大，表设计需要非常高效。

代码块

```sql
-- api_access_logs 表
CREATE TABLE api_access_logs (
    log_time TIMESTAMPTZ NOT NULL,
    api_endpoint VARCHAR(255) NOT NULL,
    http_status INT NOT NULL
);

-- 创建一个高效的索引
-- BRIN 索引在这里可能很合适，因为它适用于与物理存储顺序高度相关的时序数据
CREATE INDEX idx_logs_log_time ON api_access_logs USING BRIN (log_time);
```

```
-- 组合索引用于快速筛选特定 api 和时间
CREATE INDEX idx_logs_api_time ON api_access_logs (api_endpoint, log_time);

```

我的设计思考：

- **表分区**：对于真正的海量日志，我会按时间（比如每天或每小时）对这张表进行**分区**。这样查询时可以利用**分区裁剪**，只扫描相关的分区，性能会得到指数级提升。
- **索引选择**：BRIN 索引占空间小，对时序数据友好。而 `(api_endpoint, log_time)` 的 B-Tree 组合索引，则能高效地服务于“查询某个 API 在某个时间段内的日志”这类请求。

第二步：告警 SQL 逻辑拆解

这个需求很复杂，我决定用 CTE 一步步拆解。

1. 每分钟的错误率统计首先，我需要计算出每个 API 在每分钟的总请求数和错误请求数。

代码块

```
-- CTE 1: minute_error_rate
WITH minute_error_rate AS (
    SELECT
        -- 将时间截断到分钟
        date_trunc('minute', log_time) AS minute_window,
        api_endpoint,
        -- 计算总请求数
        COUNT(*) AS total_requests,
        -- 计算错误请求数
        COUNT(*) FILTER (WHERE http_status >= 500) AS error_requests,
        -- 计算错误率
        (COUNT(*) FILTER (WHERE http_status >= 500))::DECIMAL / COUNT(*) AS error_rate
    FROM
        api_access_logs
    WHERE
        log_time > NOW() - INTERVAL '1 hour' -- 只看最近一小时的数据
    GROUP BY
        minute_window, api_endpoint
)

```

2. 识别“连续”这是最难的一步。如何判断一个 API 在“连续的”5 分钟内都满足条件？这里有一个巧妙的技巧：用排名函数制造分组。

代码块

```sql
-- CTE 2: consecutive_check
, consecutive_check AS (
    SELECT
        *,
        -- 技巧：用当前分钟减去一个按时间排序的行号（转换成整数分钟），
        -- 对于连续的分钟，这个差值是恒定的。
        (minute_window - (ROW_NUMBER() OVER (PARTITION BY api_endpoint ORDER BY minute_window) * INTERVAL '1 minute')) AS consecutive_group
    FROM
        minute_error_rate
    WHERE
        error_rate > 0.05 -- 只看那些错误率超 5% 的分钟
)

```

3. 找到连续超过 5 分钟的组现在，我只需要在 `consecutive_check` 的结果上按 `consecutive_group` 分组，找到那些组内成员数量（即连续分钟数）大于等于 5 的即可。

代码块

```sql
-- CTE 3: final_alerting_groups
, final_alerting_groups AS (
    SELECT
        api_endpoint,
        consecutive_group,
        COUNT(*) AS consecutive_minutes,
        MIN(minute_window) AS start_time,
        MAX(minute_window) AS end_time
    FROM
        consecutive_check
    GROUP BY
        api_endpoint, consecutive_group
    HAVING
        COUNT(*) >= 5
)

```

最终查询

代码块

```sql
-- 完整查询
WITH minute_error_rate AS (
    -- ... (CTE 1)
),
consecutive_check AS (
```

```
    -- ... (CTE 2)
),
final_alerting_groups AS (
    -- ... (CTE 3)
)
SELECT * FROM final_alerting_groups;

```

我的复盘：

- 这个“连续性问题”的解法非常精妙，它将一个复杂的时间序列问题，转化成了一个分组计数问题，充分展现了窗口函数的威力。
- 整个查询虽然长，但通过 CTE 的拆解，每一步的逻辑都非常清晰。

第三步：性能优化——物化视图

上面的查询逻辑很复杂，如果每秒都去跑一遍，数据库压力会很大。这时，**物化视图 (Materialized View)** 就派上用场了。

物化视图会把查询结果**真实地存储下来**。我只需要定期刷新它，查询告警时直接查这个物化视图即可。

代码块

```
-- 创建一个物化视图，存储每分钟的错误率
CREATE MATERIALIZED VIEW mv_minute_error_rate AS
SELECT
    date_trunc('minute', log_time) AS minute_window,
    api_endpoint,
    COUNT(*) AS total_requests,
    COUNT(*) FILTER (WHERE http_status >= 500) AS error_requests
FROM
    api_access_logs
GROUP BY
    minute_window, api_endpoint;

-- 定期刷新物化视图（例如每分钟执行一次）
REFRESH MATERIALIZED VIEW mv_minute_error_rate;

```

我的收益：创建了物化视图后，我的告警查询就可以直接基于 `mv_minute_error_rate` 这个“预计算”好的结果表进行，速度会快几个数量级。

第十一周复盘

- **复杂问题的分解能力**：这是本周最大的收获。我学会了如何将一个看似无从下手的复杂需求，通过 CTE 和窗口函数，一步步分解成可执行的、逻辑清晰的 SQL 步骤。
- **性能意识**：我不再满足于“能用”，而是开始思考“好用”。在设计表结构时考虑分区和索引，在设计查询时考虑用物化视图来加速，这是从一个 SQL 用户到一个数据工程师的重要转变。
- **我的知识盲区**：对于流式计算，如果用 Flink SQL，这个问题可能会有更“实时”、更优雅的解法。比如使用 `MATCH_RECOGNIZE` 子句来做复杂事件处理（CEP）。这坚定了我后续深入学习流计算的决心。

第十二周：毕业总结：跨库差异与我的知识体系

学习目标

1. **系统梳理**：总结主流数据库（MySQL, PostgreSQL, SQL Server, Oracle）在语法、数据类型、函数等方面的核心差异。
2. **关注云原生**：再次审视云数仓（Snowflake, BigQuery）与传统数据库的设计哲学差异。
3. **构建知识体系**：形成我自己的 SQL 知识图谱和速查手册。
4. **制定后续计划**：明确下一阶段的学习目标，持续精进。

核心差异速查表：我的备忘录

特性	PostgreSQL	MySQL	SQL Server	Oracle	我的总结
自增列	`SERIAL` / `GENERATED AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(1, 1)`	`GENERATED AS IDENTITY`	语法各异，但功能相似。PG 和 Oracle 的 `GENERATED` 语法更符合 SQL 标准。
事务默认隔离级别	**读已提交 (RC)**	**可重复读 (RR)**	读已提交 (RC)	读已提交 (RC)	MySQL 是个“异类”，提供了更强的默认隔离性，但也可能带来一些额外的锁开销。
UPSERT	`ON CONFLICT DO UPDATE`	`ON DUPLICATE KEY UPDATE`	`MERGE`	`MERGE`	PG 和 MySQL 的语法更简洁，`MERGE` 功能更强大但也更啰嗦。

JSON 支持	`JSONB` **(王者)** + 丰富操作符 + GIN索引	`JSON` 类型 + `JSON_*` 函数	`JSON` 函数	`JSON` 类型 + `JSON_*` 函数	**PG 在 JSON 处理上功能最强、性能最好。**
窗口函数	非常完整	8.0 后完整支持	完整支持	完整支持	基本都已成为标配。
CTE 递归	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH` (无需 `RECURSIVE` 关键字)	`WITH`	语法略有差异，但都支持。
字符串拼接	`		`	`CONCAT()`	`+`
TOP N 查询	`LIMIT` / `FETCH FIRST`	`LIMIT`	`TOP N` / `OFFSET FETCH`	`FETCH FIRST` / `ROWNUM` (旧)	`LIMIT` 最流行，但 `FETCH` 语法是 SQL 标准。

云数仓 vs 传统数据库：设计哲学的再次碰撞

维度	传统数据库 (OLTP)	云数仓 (OLAP)
核心架构	存算一体	**存算分离**
设计目标	事务一致性、高并发 CRUD	**分析性能**、海量数据扫描
数据模型	规范化 (3NF)	**反规范化** (星型/雪花)
扩容方式	垂直扩容 (Scale-up)	**水平扩容 (Scale-out)**，计算与存储独立
成本模型	持续在线成本	**按需付费**，计算与存储分别计费
AI/ML 集成	通常通过外部应用集成	**原生集成**，用 SQL 调用模型成为趋势

我的感悟：它们不是替代关系，而是**协作关系**。OLTP 数据库作为“数据源”，负责生产和存储业务数据；云数仓作为“分析大脑”，负责对这些数据进行清洗、整合和深度挖掘，最终产出商业洞察。

我的 SQL 知识图谱 (v1.0)

我用脑图的形式，画出了我这 12 周学到的知识结构。（我会在我的笔记软件里画出这张图，核心分支包括）：

- **SQL 基础** (`SELECT`, `WHERE`, `JOIN`, `GROUP BY`)
- **高级语法** (窗口函数, CTE, `LATERAL`)

- **数据类型** (常规, `JSON`, `GIS`)
- **事务与并发** (ACID, 隔离级别, MVCC, 锁)
- **性能优化** (`EXPLAIN`, 索引, 分区)
- **数据库建模** (范式, ER图, 维度建模)
- **数据工程** (幂等, 数据质量, 流处理)
- **生态与架构** (OLTP/OLAP, 云数仓, NewSQL)

毕业，也是新的开始

12 周的学习，我完成了从“小白”到“入门者”的蜕变。我不再对 SQL 感到恐惧，而是充满了好奇和探索的欲望。我掌握了它作为一门“语言”的语法，更重要的是，我开始理解它背后所承载的“思想”——关系代数的严谨、事务处理的哲学、查询优化的艺术、数据建模的权衡。

这本日记的第一版到此结束，但我的 SQL 之旅才刚刚开始。

我的下一步学习计划：

1. **深入数据库内核**：我想了解更多关于优化器、存储引擎、锁机制的内部实现细节。
2. **Flink SQL 实战**：系统学习流式计算，并亲手搭建一个实时的应用。
3. **dbt (Data Build Tool)**：学习这个现代数据转换工具，了解如何用工程化的方式来管理数据仓库中的 SQL 转换逻辑。
4. **行业案例研究**：深入分析大厂是如何使用这些技术来解决真实业务问题的。

这本日记，我会持续更新下去。感谢这 12 周的坚持，未来可期！